

Trahēns

a privacy enabled route discovery protocol

Andrea Benetton^{1,*}

¹BlueTeam OÜ, Tallinn, Estonia

*andrea.benetton@blueteam.ee

ABSTRACT

Low-latency network-layer anonymity systems are a recent proposal for anonymous communication. They are plugged in as a service of infrastructures in next-generation network architectures which also provides ISP managed mechanisms necessary to discover the sequence of addresses from the origin to the destination to forward data packets. The approach in these architectures migrates the current Internet operating model based on Autonomous Systems (AS) and, by considering anonymity systems as a service implies that privacy is opt-in. We propose a route discovery mechanism that is privacy by design; it avoids disseminating information that can be used later to deanonymize the communications. It is trustless; no entities are required to manage routing tables, the network self-organizes.

CONFIDENTIAL

1 INTRODUCTION

CONFIDENTIAL DRAFT

2 BACKGROUND AND RELATED WORK

CONFIDENTIAL DRAFT

3 PROBLEM DEFINITION

3.1 Network Assumptions

5 Trahēns protocol operates at an equivalent to the ISO-OSI layer 3, discovering routes between two nodes connected within a graph topology. Suppose Trahēns is applied to a network communication stack, as we will take for granted in the follow. In that case, the graph could be achieved by an upper uniformity extension at the top of the underlying heterogeneous ISO-OSI layer 2 ("Data Link Layer"). This white-paper does not cover this extension (named Nexus in
10 the following), but we provide its must have.

3.2 Underlying layer extension requirements

15 Trahēns has a prerequisite that the adversary cannot attack the edges (cannot read communications in transit) but only the nodes themselves. To secure graph edges, Nexus MUST provide an encrypted, reliable, flow-controlled, full duplex connection between two adjacent node on the same underlying medium while solving all issues related to packet fragmentation.

It can be imagined as a technology agnostic generalization of MACsec[1] (IEEE 802.1AE) as it mitigates attacks directed against the physical link by defining a secure channel between peers to provide data confidentiality, data integrity, and data origin authentication.

20 Like TARANET[2] "Link Encryption and Padding", it MUST prevent an adversary from observing a data flow between two honest adjacent nodes from distinguishing chaff (used to avoid traffic analysis from actual data, control plane messages from data and different types of messages).

Also, when Nexus notify the upper layer of the availability of a new adjacent node, it MUST include the number adjacent of the connecting node, obfuscated by adding positive noise, to
25 hide the network topology. By eventually employing a private intersection set technique, Nexus also detects and remove from the counting mutual adjacent that leads to $d \leq 2$ cycles (common in a highly interconnected local topology like LAN).

3.3 Addressing schema

The upper layer, which Trahēns is part of, defines an own addressing schema.

30 Each endpoint embeds a set of long-term digital asymmetric key-pair generated locally, one for each cryptographic set supported by the node. Multiple cryptographic sets are required to move forward in managing technological obsolescence. Instead, a router rely on multiple sets of ephemeral key-pairs to encrypt and authenticate when performing routing.

A **endpoint address** is obtained by concatenating a *varint* preamble that encodes the
35 identifier of the cryptographic set used and the corresponding hash of the long term public key. A node has many node addresses as it can implement different cryptographic sets to be utilized (see section A.2); these addresses are concatenated in a **node address string**. The node address string is the information that another node must know in order to start a communication with it. When it is used outside the stack (i.e., DNS records, QRCode) a checksummed base32
40 encoding like BECH32[3] SHOULD be used.

Even if technically the same thing, the term **routing label** is used instead to indicate the hash of the ephemeral public key used for routing.

3.4 Layer communications

We can classify data exchange at layer 3 as:

- 45 • **Control plane**, ones necessary to enable the transfer of data submitted to a node and to run the network.
- **Data plane**, ones that convey data submitted to the receiver.

The purpose and the Trahēns activities make it a protocol that operates at Control Plane. Control plane protocols data unit is a **Message**.

50 A node does not send a message individually but queues them into batches of size m . As TARANET[2] setup phase, each node implements mixing before transmitting, here extended to all messages of the control plane and not just ones of the flow setup. Once there are enough messages to form a batch, the sender first performs a Fisher-Yates shuffle of them and then transmits the batch as a single **Packet**.

55 Chaffs defenses are used to defy traffic analysis attack aimed to identify communicating endpoints based on metadata such as volume, traffic patterns, and timing by adding padding traffic. This is obtained by injecting chaff messages that implement the padding of the batches in the mixing process; a pair of close coordinates the injection to maintain a constant sending rate on a edge. The rate is calculated by probing underlying Nexus layer performances.

60 This layer implements a C clock function, which assigns a natural number $C(p)$ to any packet g , a Lamport timestamp[4]) that defines an irreflexive partial ordered set of all packets circulated in the network. The timestamp is incremented before each network event, ie, the sending or receiving of a packet.

3.5 Threat Model

65 We consider an active global adversary able to force a certain number of nodes to surrender, that is, on the compromised node the adversary sees all keys and settings and has bulk access to contents and timing information of traversing protocol traffic and can also there inject, drop, delay, replay, redirect, and modify messages.

3.6 Trahēns Goals

70 **Pseudo Anonymity.** Using Pfitzmann and Köhntopp[5] formalized terminology, in the scope of functions expected by a route discovery protocol, Trahēns provides to each application "pseudo-anonymity" using a unlocalizable "initially person unlinked pseudonym" (the address) and short life "transaction pseudonym" on intermediates nodes effectively preventing the determination of the route beyond the directly adjacent nodes.

75 **Forward Authentication.** Discovering a node means not only knowing its pseudonym but exchanging respective ephemeral public keys in the process. Once a node declares its identity at the beginning of the process, a "man in the middle attack" due to the physical replacement or insertion of nodes in the path cannot be conducted.

Message Unlinkability. The messages used for exploration and the exchange of ephemeral
80 keys must possess the maximum achievable degree of bit-pattern unlinkability. Our schema guarantees message unlinkability between m_i and $m_{i \pm k}$ with $k > 2$.

Topology Concealing. To avoid the identification of the node position in the network topology by third-party nodes while maintaining the scalability of the solution, Trahēns disperses local onionized routing information in neighboring nodes and stores long-range ones in distributed
85 facilities. A node would not need to know the entire network topology because it does not use source-based routing, but a hybrid that distributes the routing information in intermediates nodes, still allowing the sender to choose the routes to use.

4 PROTOCOL DESIGN

The basic idea is to have a distributed system that carried out the process in different phases in which different nodes take on the role of sender and receiver in the transmission of control plane messages. The challenge is to preserve privacy; it is not desirable to announce exact node position to the whole network, nor scalable that any node try to collect information about all nodes of the network.

4.1 Terms and Notations

We first describe our notation.

A **node** n is any element which operates one or more associated activities related to the network layer in which Trahens operates. An **edge** is any element through which two adjacent nodes can passively send unit of information.

A node act as an **endpoint** if originates or it is the destination of the layer traffic. An exchange of messages occurs between two endpoints, initiating by the **sender** n^s and terminating at the **receiver** n^r . An endpoint transmits data originating from itself or receives data intended to him. It consumes a network service.

A **route** between n^s and n^r is a unweighted and undirected sequence of nodes whose pairwise relationships connect the two endpoints. Since there may be multiple routes from n^s to n^r through the same close with different follow up, and a node may have multiple closes each on a different route to the other endpoint, to disambiguate, a route is encoded as a tuple $(n^s, n^1, \dots, n^i, \dots, n^r)$.

Sym	Meaning	Example
$\mathcal{R}$	a cryptographically secure pseudo-random number generator	$r \leftarrow \mathcal{R}()$
$\mathcal{G}$	a key generation algorithm that on input 1^k seed outputs public key/secret key pair	$(pk, sk) \leftarrow \mathcal{G}(1^n)$
$\mathcal{MG}$	a key generation algorithm that on input 1^k seed outputs master key pair and the master chain code	$(pk, sk, cc) \leftarrow \mathcal{MG}(1^n)$
$\mathcal{CKD}$	a child key derivation algorithm that on input parent extended key k^i , parent chaincode cc^i a child index j outputs derived key k_j^{i+1} and associated chaincode cc_j^{i+1}	$(k_j^{i+1}, cc_j^{i+1}) \leftarrow \mathcal{CKD}(k^i, cc^i, j)$
$\mathcal{E}$	an encryption algorithm that on input key k and message m outputs ciphertext c	$c \leftarrow \mathcal{E}(k, m)$
$\mathcal{D}$	a decryption algorithm that on input key k and ciphertext c outputs message m	$m \leftarrow \mathcal{D}(k, c)$
$\mathcal{S}$	a signing algorithm that on input key k and message m outputs a signature s	$s \leftarrow \mathcal{S}(k, m)$
$\mathcal{V}$	a signature verification algorithm that on input key k , message m and its signature s outputs true if the signature is verified	$(true \vee false) \leftarrow \mathcal{V}(k, m, s)$
$\mathcal{H}$	a cryptographically hashing algorithm that on input message m outputs its hash h	$h \leftarrow \mathcal{H}(m)$

Table 1. Algorithms notation

A **router** node n^i is the node at a distance i from node n^s on a route. A router receives data that is not intended to it and that it forward along the route to the actual destination. It

110 provides a service to the network (i.e., other nodes). Any node can participate in the routing if it's willing to do so and is connected to at least two other nodes.

Sym	Meaning
$ n $	the degree of node n
$\langle n \rangle$	the obfuscated degree of node n (see section)
j	an iterator on a degree
k	an iterator on an obfuscated degree
z	a distance iterator
k, k^i, k_k^{i+2}	a key, the same of the i node on the route, the same for the node n^{i+2} reached along k -th edge from n^{i+1}
$k_{s,i}^{\rightarrow}, k_{k,i+2}^{\rightarrow}$	a key of the node n^s derived via $\mathcal{CKD}$ for the node n^i , the same derived for the node n^{i+2} reached along k -th edge from n^{i+1}
pk, sk	a public key, a secret key
cc, cc^i, cc_k^{i+2}	the chain code, used by $\mathcal{CKD}$ to generate the next depth level of keys, the same of the i node on the route, the same for the node n^{i+2} reached along k -th edge from n^{i+1}
w, w_k^{i+2}	a random number used as index to generate keys in $\mathcal{CKD}$, the same for the node n^{i+2} reached along k -th edge from n^{i+1}
ed, ed_k^{i+2}	the corresponding w encrypted
p^i	an unencrypted payload generated at node n^i
e^i	an encrypted payload generated at node n^i
s^i	a signature generated at node n^i
L, lpk, lsk, led	the left label and the left pk sk and ed.
R, rpk, rsk, red	the right label and the right pk sk and ed.

Table 2. Symbols in the 5.1 section

The **distance** d is the number of hops between two nodes n^s and n^r on route. A **neighbor** is a node within a defined distance from n^s . A **close** is the node n^1 at a distance 1 from node n^s .

115 The **degree** of a node $|n|$ is the number of closes connected with n that is, the number of edges connected to it. A **leaf** node is a node with degree 1, typically an endpoint application running on a device.

The keywords “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “NOT RECOMMENDED”, “MAY”, and “OPTIONAL”
120 in this document are to be interpreted as described in BCP 14 namely RFC2119[6] and RFC8174[7] when, and only when, they appear in all capitals, as shown here.

Message Sequence Charts (MSC) format is described in ITU Z.120[8] recommendation.

Packet diagrams in this document use the format described in Section 3.1 of RFC2360[9].

4.2 Protocol Phases

125 The **flooding phase**(5.1) operates in the neighborhood of the originator node and keeps anonymized information about routes between to **Gateways** in a distributed and redundant form. A gateway is a node that when presented with the hash of the address of the originator node, it will forward it through an anonymous route. The phase starts when Nexus of the node n^s notifies Trahēns that a new edge is available and then is repeated periodically; the
130 originator reaches out gateways along the new branch of the network reachable through that

edge, a process that takes place symmetrically in opposite directions on both closes of an edge when it is established.

In the **registration phase**(5.2), the gateway files own label and the hash of the initiator's address on Beacons known to him. A **Beacon** is a node on which a B-facility runs, that is, simplifying a database table that accepts updates to a record by providing proof of ownership of the same and that accepts queries to the double-hash of originator's address. Statistically, while the topology near the originators can change often, beacons, as any network service provider, are of high availability and relative stability in the local topology. Up to this point the information on how to reach the originator is only available in nodes that know a beacon that has the necessary record.

In the subsequent **propagation phase**(5.3), the beacon posts its address and the triple-hash of originator's address to an authority node, in which runs the A-facility. If it is authoritative for the triple-hash of originator's address then it proceed to register else it forward the request to others known authorities. Being authoritative of address space means that it holds a list of originators which 3address matches a bit pattern through a bit-wise AND-masking. The masking depth, namely the number of bits not set in the mask, partition the address space and therefore defines the memory and computational resources needed by the A-Facility.

At last the **resolving phase**(5.4) is executed when a node have to retrieve the route to a destination. The querying node first checks that a destination is between neighboring nodes, that is those identified during the execution of a flooding phase. If not, contact your beacons to find out if and which gateways they registered to lead to the destination. In the absence of feedback, the querying node asks beacons for the list of available authorities. If it does not find any authoritative, then it queries the non-authoritative ones for the authoritative ones. From there, they obtain the address of the beacons having gateways registered there that lead to the destination.

5 PROTOCOL DETAILS

This section presents the details of Trahēns message formats and processing functions. We show how to create messages that fulfill design goals on the sender, how intermediate nodes process these messages, and how Beacon nodes construct the reply.

5.1 Flooding neighborhood to find Beacon and Gateways

As the sender n^s knows nothing about the topology reachable through the new edge, as soon as the CLOSEADDED event is raised, it floods its neighbor nodes along the new edge till d_{max} with T-FLOOD messages. When a T-FLOOD message is received by a node, it computes a route label L^i , that is the hash of an ephemeral public key generated at n^i ; this label is forwarded to all n^{i+1} in the flooding process. By repeating this subsequently on each node of the route, i -th node can address a message to head back to the sender n^s . For this reason we call it the **left** label of the route at i -th node. This label is kept in an entry of the routing table **RouteT** of a node (see table 3); if the OutLabel is set to 0 then any incoming message headed to L is arrived back at the original sender.

Field	T-FLOOD	T-ACK-L	Description
Node	n^{i-1}	n^{i-1}	The edge on which message come from
InLabel	L^i	R^{i+1}	The label of the route in the incoming message
OutLabel	L^{i-1}	R^i	The label of route in the outgoing message
OriginPK	$pk^{\vec{s},i}$	$pk^{\vec{r},i}$	The child public key of message originator derived to be used on i^{th} node
Edge	led^i	red^i	A unique identifier of the outgoing edge
Created	$T_{created}$	$T_{created}$	The creation timestamp of the entry

Table 3. RouteT table fields on node n^i

The same process of label creation happens when the message T-ACK-L come back from the receiver to the sender. A label R^i is created at each hop, called the **right** label of the route at i -th node. The sender is now able to send a message to a replying node.

Once the T-FLOOD packet has been received by a gateway or a beacon, on each intermediate node, have been defined all L labels necessary for a receiver to send back a message to the sender. Similarly, when the sender get back the T-ACK-L message, on each intermediate node have been defined all R labels required for a sender to send a message to the receiver .

Thirdly, the sender and the recipient must exchange public keys so that, once a node has declared its identity at the beginning of the session, the exchange of information is encrypted and authenticated. It is not possible for the intermediates nodes to retransmit the same sender key because messages along that route discovery would have a common bit pattern, breaking bitwise unlinkability. To solve it, at each hop we should generate a child public key $lpk^{\vec{s},i}$ from the sender key lpk^s according to the deterministic schema provided by the Bitcoin Improvement proposal 32[10] using the HMAC-SHA512 function as specified in RFC4231[11]. The schema can eventually be adapted for Curve25519[12]. All keys are non-hardened for the sake of this document.

To still apply the randomization technique described in the original paper, the node n^i have to know how many messages are needed by n^{i+1} to be sent to n^{i+2} . The retrieving of this information from the close n^{i+1} is performed by the underlying Nexus and exposed in the CLOSEADDED event as the obfuscated degree (**ODegree**), hereafter with the notation $\langle n^{i+1} \rangle$. Nexus also detects and doesn't expose mutual closes that leads to $d \leq 2$ cycles and add a positive noise to hide the network topology.

Hence, the node n^i creates a set of tuple $lpk^{s \rightarrow i+2}, lcc^{i+2}$ of cardinality $\langle n_{i+1} \rangle$, using a random number as child index for the public key derivation function $\mathcal{CKD}_p$. The node n^{i+1} performs a shuffle on this set before the forwarding, resulting in further hiding the following topology.

195 Furthermore the message from n^i to n^{i+1} doesn't include lcc^{i+1} so it's impossible for n^{i+1} to generate keys for n^{i+2} .

Field	Value	Description
Label	L^i	The label of a route at the given node
SK	sk^i	The private key of n^i
PK	pk^i	The public key of n^i
Created	$T_{created}$	The creation timestamp of the entry

Table 4. NodeT table fields on node n^i

Algorithm 1 describe how the elements necessary to build the starting T-FLOOD message are computed on node n^s when the event CLOSEADDED is raised.

The originating node performs the master key generation function $\mathcal{MG}()$ to generate the master key pair pk^s and sk^s and the master chain code cc^s .

With these it can use $\mathcal{CKD}_p(lp k^s, lcc^s, j)$ function to computes the child extended public key $lp k^{s,1}$ and the child chaincode cc^1 from the parent extended public key $lp k^s$ and parent lcc^s for the new edge j .

It also generates the label L^s by hashing $\mathcal{H}(lp k^s)$ which becomes the route local label on that edge for the sake of routing back any message to n^s from n^1 .

Algorithm 1 On sender node n^s , start T-FLOOD direct propagation.

Require: $\langle n^1 \rangle$, the ODegree of the next node

```

1: on CLOSEADDED( $n^1, \langle n^1 \rangle$ ) do
2:   if  $\langle n^1 \rangle > 0$  then
3:      $(lp k^s, lsk^s, lcc^s) \leftarrow \mathcal{MG}()$ 
4:      $d \leftarrow 1$ 
5:      $w^1 \leftarrow \mathcal{R}()$ 
6:      $(lp k^{s,1}, lcc^1) \leftarrow \mathcal{CKD}_p(lp k^s, lcc^s, w^1)$ 
7:      $led^1 \leftarrow \mathcal{E}(lp k^s, w^1)$ 
8:      $L^s \leftarrow \mathcal{H}(lp k^s)$ 
9:     Insert in RouteT  $[0, L^s, 0, 0, 0, T_{now}]$ 
10:    Insert in NodeT  $[L^s, lsk^s, lcc^s, T_{now}]$ 
11:    for  $k \leftarrow 1, \langle n^1 \rangle$  do
12:       $w_k^2 \leftarrow \mathcal{R}()$ 
13:       $(lp k_k^{s,2}, lcc_k^2) \leftarrow \mathcal{CKD}_p(lp k^{s,1}, lcc^1, w_k^2)$ 
14:       $led_k^2 \leftarrow \mathcal{E}(lp k^s, w_k^2)$ 
15:    end for
16:     $p^s \leftarrow (d \| lp k^s \| lp k^{s,1} \| led^1 \| (lp k_1 \| \dots \| lp k_{\langle n^1 \rangle})^{s,2} \| (lcc_1 \| \dots \| lcc_{\langle n^1 \rangle})^2 \| (led_1 \| \dots \| led_{\langle n^1 \rangle})^2)$ 
17:     $s^s \leftarrow \mathcal{S}(lsk^s, \mathcal{H}(p^s))$ 
18:    Send to  $n_1$ :  $(p^s, s^s)$ 
19:  end if
20: end on

```

Algorithm 2 describes what happens on any node n^i upon the reception of the message from the close n^{i-1} . The message contains the label L^{i-1} , the public key $lpk^{s \rightarrow i}$ of n^s as seen by n^i , and a set of tuples of public key and chaincode, each one to be sent by n^{i+1} with a message to a close except the originating one. The set is shuffled before the forwarding so the previous
210 generating hop n^{i-1} cannot do any assumptions on which key go to each node.

Similarly to what we saw at the sender, the intermediate node calculate the label L^i such any message that contain it incoming from node n^{i+1} can be forwarded to n^{i-1} headed to L^{i-1} . For each edge, except the originator, the process forward the flood, generating the public keys $lpk^{s \rightarrow i+2}$ and inserting an ephemeral entry $(L^i, L^{i-1}, L^{i+1}, lpk^{s \rightarrow i}, keyseed^i, T_{eph})$.

215 T_{eph} is a short-term deadline after which the entry will be removed from the table. The life of the entry will be extended if an T-ACK-L eventually come back along the route.

The node creates and includes in each message also a set of child public key of the sender $lpk^{s \rightarrow i+2}$ and related chaincodes lcc^{i+2} that the node n^{i+1} have to provide to each of his n^{i+2} close.

Algorithm 2 On intermediate node n^i , T-FLOOD direct propagation.

Require: $\langle n_j^{i+1} \rangle$, the ODegree of the next node along the edge of index j

```

1: on reception from  $n^{i-1} : (p^{i-1}, s^{i-1})$  do
2:    $p^{i-1} \rightarrow (d, lpk^{i-1}, lpk^{s,i}, led^i, (lpk_1, \dots, lpk_{\langle n_j^i \rangle})^{\overrightarrow{s,i+1}}, (lcc_1, \dots, lcc_{\langle n_j^i \rangle})^{i+1}, (led_1, \dots, led_{\langle n_j^i \rangle})^{i+1})$ 
3:   if  $\mathcal{V}(lpk^{i-1}, p^{i-1}, s^{i-1})$  then
4:      $L^{i-1} \leftarrow \mathcal{H}(lpk^{i-1})$ 
5:     if  $d = d_{max}$  or  $\langle n^i \rangle = 0$  then
6:       Insert in RouteT  $[n^{i-1}, 0, L^{i-1}, lpk^{s,i}, led^i, T_{now}]$ 
7:     else if  $d < d_{max}$  then
8:        $(lpk^i, lsk^i) \leftarrow \mathcal{G}()$ 
9:        $L^i \leftarrow \mathcal{H}(lpk^i)$ 
10:      Insert in RouteT  $[n^{i-1}, L^i, L^{i-1}, lpk^{s,i}, led^i, T_{now}]$ 
11:      Insert in NodeT  $[L^i, lsk^i, 0, T_{now}]$ 
12:      Shuffle  $\{(lpk, lcc, led)_1, \dots, (lpk, lcc, led)_{\langle n^i \rangle}\}^{\overrightarrow{s,i+1}}$ 
13:      for  $j \leftarrow (1 \dots |n^i|), n_j^i \neq n^{i-1}$  do
14:         $p_j^i \leftarrow (d + 1 \| lpk^i \| lpk_j^{s,i+1} \| led_j^{i+1})$ 
15:        if  $\langle n_j^{i+1} \rangle > 0$  and  $d < (d_{max} - 1)$  then
16:          for  $k \leftarrow 1, \langle n_j^{i+1} \rangle$  do
17:             $w_k^{i+2} \leftarrow \mathcal{R}()$ 
18:             $(lpk_k^{s,i+2}, lcc_k^{i+2}) \leftarrow \mathcal{CKD}_p(lp_k^{s,i+1}, lcc^{si+1}, w_k^{i+2})$ 
19:             $led_k^{i+2} \leftarrow \mathcal{E}(lp_k^{s,i}, w_k^{i+2})$ 
20:          end for
21:           $p_j^i \leftarrow p_j^i \| (led_1 \| \dots \| led_{\langle n_j^{i+1} \rangle})^{i+2} \| (lpk_1 \| \dots \| lpk_{\langle n_j^{i+1} \rangle})^{\overrightarrow{s,i+2}} \| (lcc_1 \| \dots \| lcc_{\langle n_j^{i+1} \rangle})^{i+2}$ 
22:        end if
23:         $s_j^i \leftarrow \mathcal{S}(lsk^i, \mathcal{H}(p_j^i))$ 
24:        Send to  $n_j^i$ :  $(p_j^i, s_j^i)$ 
25:      end for
26:      if  $(n^i \text{ is Gateway}) \vee (n^i \text{ is Beacon})$  then
27:        raise SENDANSWER( $L^{i-1}$ )
28:      end if
29:    end if
30:  end if
31: end on

```

Algorithm 3 On receiver node n^r , start of T-ACK-L reverse propagation.

```

1: procedure SENDANSWER( $L^{r-1}$ )
2:    $(n^{r-1}, lpk^{\overrightarrow{s}, \overrightarrow{r}}, led^r) \leftarrow$  for RouteT.OLabel= $L^{r-1}$ 
3:    $(rpk^r, rsk^r, rcc^r) \leftarrow \mathcal{MG}()$ 
4:    $w^{r-1} \leftarrow \mathcal{R}()$ 
5:    $(rpk^{\overrightarrow{r}, \overrightarrow{r-1}}, rcc^{r-1}) \leftarrow \mathcal{CKD}_p(rpk^r, rcc^r, w^{r-1})$ 
6:    $red^{r-1} \leftarrow \mathcal{E}(rpk^r, w^{r-1})$ 
7:    $R^r \leftarrow \mathcal{H}(rpk^r)$ 
8:   Insert in RouteT  $[0, R^r, 0, 0, 0, T_{now}]$ 
9:   Insert in NodeT  $[R^r, rsk^r, rcc^r, T_{now}]$ 
10:   $w^{r-2} \leftarrow \mathcal{R}()$ 
11:   $(rpk^{\overrightarrow{r}, \overrightarrow{r-2}}, rcc^{r-2}) \leftarrow \mathcal{CKD}_p(rpk^{\overrightarrow{r}, \overrightarrow{r-1}}, rcc^{r-1}, w^{r-2})$ 
12:   $red^{r-2} \leftarrow \mathcal{E}(rpk^r, w^{r-1})$ 
13:   $p^r \leftarrow (L^{r-1} \parallel d \parallel rpk^r \parallel rpk^{\overrightarrow{r}, \overrightarrow{r-1}} \parallel red^{r-1} \parallel rpk^{\overrightarrow{r}, \overrightarrow{r-2}} \parallel rcc^{r-2} \parallel red^{r-2})$ 
14:   $e^r \leftarrow \mathcal{E}(lpk^{\overrightarrow{s}, \overrightarrow{r}}, ("Rcver" \parallel d \parallel \mathcal{H}(n^r) \parallel rpk^r))$ 
15:   $s^r \leftarrow \mathcal{S}(rsk^r, \mathcal{H}(e^r))$ 
16:  Send to  $n^{r-1}$ :  $(p^r, e^r, s^r)$ 
17: end procedure

```

The relay back of T-ACK-L is described in algorithm 4 the message is conveyed back to the originating node n^s .

The key $lpk^{s \rightarrow i}$ is used to onion encrypting information contained in a route-back message T-ACK-L so that the ODegree of n^i required by n^s to efficiently build the route is not disclosed to any intermediate node.

Algorithm 4 Intermediate node n^i , reverse propagation of T-ACK-L.

```

1: on reception from  $n^{i+1} : (p^{i+1}, e^{i+1}, s^{i+1})$  do
2:    $p^{i+1} \rightarrow (L^i, d, rpki^{i+1}, rpkr,i, red^i, rpki,i-1, rcc^{i-1}), red^{i-1}$ 
3:   if  $\mathcal{V}(rpki^{i+1}, e^{i+1}, s^{i+1})$  then
4:      $d \leftarrow d - 1$ 
5:     if  $d \leq d_{max}$  then
6:        $(L^{i-1}) \leftarrow \text{for RouteT.InLabel} = L^i$ 
7:        $(led^i) \leftarrow \text{for NodeT.Label} = L^i$ 
8:       if  $\exists L^{i-1}$  then
9:          $R^{i+1} = \mathcal{H}(rpki^{i+1})$ 
10:        if  $L^{i-1} \neq 0$  then
11:           $(rpki^i, rski^i) \leftarrow \mathcal{G}()$ 
12:           $R^i \leftarrow \mathcal{H}(rpki^i)$ 
13:           $w^{i-2} \leftarrow \mathcal{R}()$ 
14:           $(rpkr,i-1, rcc^{i-1}) \leftarrow \mathcal{CKD}_p(rpki^i, rcc^i, w^{i-2})$ 
15:           $red^{i-2} \leftarrow \mathcal{E}(rpkr,i, w^{i-2})$ 
16:          Insert in RouteT  $[n^{i+1}, R^i, R^{i+1}, rpkr,i, red^i, T_{now}]$ 
17:          Insert in NodeT  $[R^i, rpki^i, rski^i, 0, T_{now}]$ 
18:           $(lpk^{s,i}, led^i) \leftarrow \text{for NodeT.Label} = L^i$ 
19:           $p^i \leftarrow (L^{i-1} \parallel d \parallel rpki^i \parallel rpkr,i-1 \parallel red^{i-1} \parallel rpki,i-2 \parallel rcc^{i-2} \parallel red^{i-2})$ 
20:           $e^i \leftarrow \mathcal{E}(lpk^{s,i}, ("INode" \parallel d \parallel lpki^i \parallel led^i \parallel e^{i+1}))$ 
21:           $s^i \leftarrow \mathcal{S}(rski^i, \mathcal{H}(e^i))$ 
22:          Send to  $n^{i-1} : (p^i, e^i, s^i)$ 
23:        else if  $d = 0$  and  $L^{i-1} = 0$  then
24:          Insert in RouteT  $[n^{i+1}, 0, R^{i+1}, rpkr,i, red^i, T_{now}]$ 
25:          if  $\text{PROCESSANSWER}(L^i, e^{i+1}) = \text{success}$  then
26:            SENDCONFIRMATION( $R^{i+1}, \mathcal{E}(rpkr,i, ("Rcver" \parallel d \parallel \mathcal{H}(n^s) \parallel lpk^s))$ )
27:          end if
28:        end if
29:      end if
30:    end if
31:  end if
32: end on

```

225 In sync with receiving T-ACK-L messages, the sender populates a table of possible routes (see algorithm 5).

Algorithm 5 Sender s , acknowledge reception processing.

```

1: function PROCESSANSWER( $L^s, e^1$ )
2:    $(lsk^s, lcc^s) \leftarrow$  for NodeT.Label= $L^s$ 
3:   for  $z \leftarrow 1, d_{max}$  do
4:      $(pre^z, d_{exp}^z, lpk^z, p[...]) \leftarrow \mathcal{D}(lsk^{\overrightarrow{s, z-1}}, e^z)$ 
5:     if  $d_{exp}^z \neq z$  then
6:       return failure
7:     end if
8:     if  $pre^z = \text{"Rcver"}$  then
9:        $p[...] \rightarrow n^z$ 
10:      Insert in NeighborT [ $n^z, R^1, \{((lsk^{\overrightarrow{s, 1}}, lpk^1, led^1), \dots, (lsk^{\overrightarrow{s, z}}, lpk^z, led^z))\}, T_{now}$ ]
11:      return success
12:     else if  $pre^z = \text{"INode"}$  then
13:        $p[...] \rightarrow (led^z, e^z)$ 
14:        $w^z \leftarrow \mathcal{D}(lsk^{\overrightarrow{s, z}}, led^z)$ 
15:        $(lsk^{\overrightarrow{s, z}}, lcc^z) \leftarrow \mathcal{CKD}_s(lsk^{\overrightarrow{s, z-1}}, lcc^{z-1}, w^z)$ 
16:     else
17:       return failure
18:     end if
19:   end for
20: end function

```

The last step provided in algorithm 4 is the sending of an end to end encrypted T-ACK-R message so receiver is aware that the T-ACK-L is effectively received by sender.

230 From the moment n^r receives T-ACK-R, that contains n^s node address and public key, it is therefore possible to establish a bidirectional and encrypted communication with a gateway or a beacon. By doing ECDH, we can use symmetric encryption for better performance.

5.2 Registering a route

235 The last step of this phase consists in registering gateways in beacons. Each gateway discovered, when receives T-ACK-R, sends to every known beacon a GATEWAY-REGISTER message consisting in the concealed address of the other end of the route n^s and in the signature of the n^s plain address done by the same node.

To go further in the analysis of following steps, we brief about the facilities involved in the process.

240 Beacon nodes run B-Facility; its purpose is to run a process to gather information on the neighborhood topology proactively and to keep data updated in the **authority** nodes.

It is not necessary that all nodes run facilities; it is only necessary that there is a certain "density" of nodes running each of them within the network.

A B-Facility maintains a set of tables:

- a gateways table (**GTable**) that contains a list of gateways to a given nodes present in its neighbor,
- an authorities table (**ATable**) of authority nodes and their authoritative address spaces.

When a beacon receives the GATEWAY-REGISTER message, it looks up in the ATable for authority nodes that are authoritative for that address. Authoritative means that authority has previously advertised the network via an AUTHORITY-REGISTER message that it holds

250 a list of gateway with hash of the address matches the pattern field contained in the message through a bitwise AND-masking. In this way, each authority is configured to be authoritative of a subset of node address namespace. The **masking depth**, namely the number of bits not set (0) in the mask, defines the memory and computational resources needed by the authority.

The beacon send to each authority, that is authoritative for the hash of the gateway address,
255 a GATEWAY-ENLIST message.

An authority maintains a set of tables:

- a gateways table (**GTable**) that contains a list of gateways to a given nodes present in its neighbor,
- an authorities table (**ATable**) of authority nodes and their authoritative address spaces.

260 5.3 Propagate

5.4 Resolving an address in a sequence of intermediate nodes

1. notifying the authoritative TD-Nodes known when the address of a node that respects this form is registered with a TD-Node.
2. making an authoritative TD-Node notify the other known authoritative TD-Nodes when
265 a mask-compliant address is registered.

A ANNEXES

A.1 Variable length unsigned integer

Bytes are a MSB base-128 encoding of the number. The high bit in each byte signifies whether another digit follows. To make sure the encoding is one-to-one, one is subtracted from all but the last digit. Thus, the byte sequence $a[]$ with length len , where all but the last byte has bit 128 set, encodes the number:

$$(a[len - 1] \& 0x7F) + \sum_{i=1}^{len-1} (128^i \cdot (a[len - i - 1] \& 0x7F) + 1)) \quad (1)$$

Properties:

- Very small (0-127: 1 byte, 128-16511: 2 bytes, 16512-2113663: 3 bytes)
- Every integer has exactly one encoding
- Encoding does not depend on size of original integer type
- No redundancy: every (infinite) byte sequence corresponds to a list of encoded integers.

0:	[0x00]
1:	[0x01]
127:	[0x7F]
128:	[0x80 0x00]
255:	[0x80 0x7F]
256:	[0x81 0x00]
16383:	[0xFE 0x7F]
16384:	[0xFF 0x00]
16511:	[0xFF 0x7F]
65535:	[0x82 0xFE 0x7F]
2^{32} :	[0x8E 0xFE 0xFE 0xFF 0x00]

Refer to [13] for a C++ implementation.

A.2 Candidate Cryptosystems

In this document, it's not necessary to make any final choice regarding cryptographic systems, but some points may already be highlighted.

Routers often embed part of the network protocols, adopting hardware solutions to accelerate the processing of the packets. Cryptography algorithms are exemplary candidates for this optimization. Therefore the design of a TCP-IP replacement poses the technological challenge compared to implementations that are thought to be only software regarding the hardware higher inertia to change.

Although one cryptographic system may reasonably be preferred today rather than another, no one can exclude that a vulnerability can be identified in the future. In particular, the approach of quantum computing represents a real threat that does not allow choices made now and forever.

For these reasons, the implementation of Nova will probably have to offer from the beginning at least two non-quantum resistant sets, each one containing for the same function a different cryptographic algorithm. The initial handshake of the related stratum communication let's two nodes agree which set to use.

As the purpose of this paper is to demonstrate the consistency and feasibility of Nova stack, in the following of this paper we assume the use of "Set 1" for examples.

Function	Set 1	Set 2
Key Exchange Scheme	DH	DH
Asymmetric Cryptography Algorithm	X25519	Secp256k1
Authenticated Encryption with Associated Data	AES_256_GCM[14]	CHACHA20_POLY1305[15]
Hash function	SHA3-256	BLAKE2s

Table 5. Nova suggested initial cryptographic sets

A.3 Packet header specification

This is the packet header specification:

Ver (VarUInt) Each sent packet starts with a VarUInt, which indicates the version of the header specifications used. A version > 0x00 suggests that the node has deviated from the specification within this document. A node MUST ignore packets with an unknown version.

Timestamp (ULong) The Lamport timestamp.

Number (VarUInt) The number of messages.

Message 0

Type (VarUInt) The type of message. Refer to table 6 for messages related to Trahēns . A node MUST ignore packets with an undefined class for the given version.

Length (VarUInt) The length of message in bytes. It is necessary to skip in the parsing unrecognized messages.

Message 1 (...)

VarUInt format is summarized in annex A.1.

Message Types	Message Types name	See section
0x00	DATAFLOW	
0x01	CHAFF	
0x20	T-FLOOD	5.1
0x21	T-ACK-L	5.1
0x22	T-ACK-R	

Table 6. Message Types

References

- [1] “802.1ae: MAC Security (MACsec) |.” [Online]. Available: <https://1.ieee802.org/security/802-1ae/>
- [2] C. Chen, D. E. Asoni, A. Perrig, D. Barrera, G. Danezis, and C. Troncoso, “TARANET: Traffic-Analysis Resistant Anonymity at the Network Layer,” in *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*. London: Ieee, Apr. 2018, pp. 137–152. [Online]. Available: <https://ieeexplore.ieee.org/document/8406596/>
- [3] P. Wuille and G. Maxwell. (2017) Base32 address format for native v0-16 witness outputs. [Online]. Available: <https://github.com/bitcoin/bips/blob/master/bip-0173.mediawiki>

- [4] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 7 1978. [Online]. Available: <http://doi.acm.org/10.1145/359545.359563>
- [5] A. Pfitzmann and M. Köhnemann, "Anonymity, unobservability, and pseudonymity—a proposal for terminology," in *Designing privacy enhancing technologies*. Springer, 2001, pp. 1–9.
- [6] S. Bradner, "Key words for use in rfcs to indicate requirement levels," Internet Requests for Comments, RFC Editor, Bcp 14, Mar. 1997. [Online]. Available: <http://www.rfc-editor.org/rfc/rfc2119.txt>
- [7] B. Leiba, "Ambiguity of uppercase vs lowercase in rfc 2119 key words," Internet Requests for Comments, RFC Editor, Bcp 14, May 2017. [Online]. Available: <http://www.rfc-editor.org/rfc/rfc8174.txt>
- [8] ITU-T, "Z.120 : Message Sequence Chart (MSC)," ITU-T, Recommendation Z.120, Feb. 2011. [Online]. Available: <https://www.itu.int/rec/T-REC-Z.120-201102-I/en>
- [9] G. Scott, "Guide for internet standards writers," Internet Requests for Comments, RFC Editor, BCP 22, Jun. 1998. [Online]. Available: <http://www.rfc-editor.org/rfc/rfc2360.txt>
- [10] P. Wuille. (2012, 02) Bitcoin improvement proposal 32: Hierarchical deterministic wallets. [Online]. Available: <https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki>
- [11] M. Nystrom, "Identifiers and test vectors for hmac-sha-224, hmac-sha-256, hmac-sha-384, and hmac-sha-512," Internet Requests for Comments, RFC Editor, RFC 4231, 12 2005. [Online]. Available: <http://www.rfc-editor.org/rfc/rfc4231.txt>
- [12] D. Khovratovich and J. Law, "Bip32-ed25519: Hierarchical deterministic keys over a non-linear key space," in *2017 IEEE European Symposium on Security and Privacy Workshops (EuroS PW)*, 04 2017, pp. 27–31.
- [13] "bitcoin/serialize.h at v0.13.2 · bitcoin/bitcoin," <https://github.com/bitcoin/bitcoin/blob/v0.13.2/src/serialize.h#L307L372>, (Accessed on 10/11/2019).
- [14] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac," NIST Recommendation, NIST, NIST Pub 800-38D, 11 2007. [Online]. Available: <https://www.nist.gov/publications/recommendation-block-cipher-modes-operation-galoiscounter-mode-gcm-and-gmac>
- [15] Y. Nir and A. Langley, "Chacha20 and poly1305 for ietf protocols," Internet Requests for Comments, RFC Editor, RFC 7539, May 2015. [Online]. Available: <http://www.rfc-editor.org/rfc/rfc7539.txt>

Acknowledgements (not compulsory)

Acknowledgements should be brief, and should not include thanks to anonymous referees and editors, or effusive comments. Grant or contribution numbers may be acknowledged.

Author contributions statement

Must include all authors, identified by initials, for example: A.A. conceived the experiment(s), A.A. and B.A. conducted the experiment(s), C.A. and D.A. analysed the results. All authors reviewed the manuscript.

Additional information

To include, in this order: **Accession codes** (where applicable); **Competing financial interests** (mandatory statement).

The corresponding author is responsible for submitting a competing financial interests statement on behalf of all authors of the paper. This statement must be included in the submitted article file.

CONFIDENTIAL DRAFT